Risk — Software Design

Document ID	SDS-RISK-001
Revision	dfa6eda
Date	2026-11-10
Status	Controlled in git

Contents

1 Purpose 3
2 Design Outputs 3
3 Dependencies 5
4 Out of scope 5

Id: `SDS-RISK-001` · Context: `risk`

1 Purpose

This context owns the risk register as part of the record, and the risk acceptability criteria the register is evaluated against. Its language is risk analysis and risk evaluation: a risk is a hazard, the hazardous situation it leads to and the harm that situation can cause — one harm pathway per risk — with a severity from the harm and a probability from the situation; a safety risk or a STRIDE threat; the risk controls that reduce it, each a requirement; the initial risk and the residual risk, the same estimate before and after the controls; and a proposal, a rating or a policy no person has approved yet. The method (hazard → situation → harm; evaluate, control, verify, evaluate again) is the team's; this context keeps the register and checks what a machine can check.

2 Design Outputs

Its one component, and how the components of other contexts use it:

Component	Responsibility	Meets
Risk register	Reads the declared risk policy (refusing a second) and the register, evaluates each risk's initial and residual risk against the policy, gives each risk its residual decision, and writes the risk rules as findings, each blocking or a warning and tied to the risk it is about	item-02-001, item-02-002, item-02-003

The register is the frontmatter of `kind: risk` documents — the record interface authors write, with one entry per risk:

```yaml
risks:
  - id: RISK-TOOL-001
    category: safety            # or security, with stride:
    linked: [RISK-TOOL-002]     # optional: risks it bears on
    hazard: "What could go wrong, and the sequence of events"
    situation: "The hazardous situation"
    harm: "Who is hurt, and how"
    severity: Serious           # from the harm
    probability: Possible       # from the situation
    level: Medium               # optional; must be the policy's level
    controls: [item-04-001]     # each control is a requirement of the spec
    residual: {probability: Rare}  # severity optional: the initial's
    acceptance: {by: "...", rationale: "..."}
    status: proposed            # until a person approves the rating
```

The rules the Risk register applies, which a reviewer needs to judge it:

- **The policy.** The project declares at most one risk policy (a `risk_policy` in the frontmatter of a record document; the first by path is the one read). A policy that does not give a level for every pair, or an acceptability for every level, is one blocking finding naming its document, and the register is not checked further until it is fixed. The policy's status is its document's; an unapproved policy is a warning. A policy that lists a severity or a probability twice is malformed: one pair would have two levels.
- **Evaluation.** With no policy no risk is evaluated, and a register with risks blocks. A risk with no controls keeps its initial risk as its residual risk, whatever residual it declares: only a control reduces a risk. A hazard, situation or harm is text; anything else is empty. A control or a link written as one value, not a list, is that one value. The policy's severities may be numbers as well as names. A risk's status is its own, else its document's, else *proposed*: no status is never read as approved. A risk id is text or a number; anything else is no id. A recorded `level`, initial or residual, must be the policy's.
- **The residual decision** is one of: *not evaluated* (no policy, no residual level, or a control without a passing test), *acceptable*, *accepted* (a level the policy accepts only with justification, with who accepted it and why, both text), *needs acceptance*, or *unacceptable*. Only acceptable and accepted let a release through.
- **The register.** A document of kind `risk` (in any case) holds its risks as a list of mappings under `risks`; any other shape, or no `risks` at all, is a blocking finding naming the document, never a register silently left out. A document that only declares the risk policy is not a register.
- **The findings** are every refusal of item-02-002 and item-02-003, and also a risk whose status is neither proposed nor approved, which blocks. A proposed risk or policy is a warning. The Risk register is the one place these rules are written.

Relationships, in the direction of the arrow:

- The **Release gate** (`release`) reports the risk findings of the Risk register: it adds the blocking findings to its blocking list and the warnings to its warnings: the part of item-02-002 and item-02-003 the release context applies.
- The **Projection** (`graph`) reads risks and findings with the Risk register — the policy, the register, each residual decision and the findings — into the risks named graph for its gate shapes (item-06-008, owned by `graph`), so the release gate and the shapes cannot disagree.
- The **Verification report** (`publishing`) reads risk status and residual decisions with the Risk register, beside the requirements that control each risk.

The Risk register is handed the requirement ids and the verified ones; it reads nothing of the spec itself. It parses frontmatter with the shared kernel.

Assumption: one harm pathway per risk, with one probability; no control effect is recorded apart — the residual is all the controls folded together. The register is kept this small; each can grow later without changing a register written to this format.

3 Dependencies

Layer 1 of the dependency rule, a leaf: it depends only on the shared kernel, and on no other context. The contexts that use it sit above it: `release` (the release gate), `publishing` (the verification report) and `graph` (the projection).

4 Out of scope

Whether a risk control is effective is shown in the record only as far as a machine can see it: the control is verified and the risk, evaluated again, has an acceptable or accepted residual. Whether the control does what it claims in the code, whether the rating is right, and whether a residual is as low as reasonably practicable stay human judgements, made in review and recorded by approving the rating (`status: approved`).